Chat Application

Using Python 3 Socket Programming and Cisco Packet Tracer

Department of Computer Science & Engineering

Bachelor of Technology — Computer Networks

Academic Year 2025-26

PROJECT BASED LEARNING

Project No. 01

Subject: Computer Networks (CN) — 4th Semester

Project Title: Chat Application — Client Server Architecture

Language: Python 3 (socket and threading modules)

Simulation Tool: Cisco Packet Tracer 8.x

Submitted To: Department of Computer Science & Engineering

Made with GAMMA

Abstract

This project presents the design and implementation of a Chat Application using Python 3. The application is based on the Client-Server architecture and makes use of TCP socket programming for real-time message exchange between multiple users. On the network side, Cisco Packet Tracer 8.x has been used to simulate the physical and logical network topology, including IP address assignment, switch configuration, and packet-level simulation using PDU (Protocol Data Unit) tools.

The project was implemented using only the built-in Python libraries — socket and threading — which means no external packages need to be installed. The server program manages all incoming connections, handles multiple clients simultaneously using threads, and broadcasts messages to all connected users. The client program connects to the server and allows a user to send and receive messages in real time.

Made with GAMMA

Introduction

Communication over a network is one of the most fundamental applications of Computer Networks. A Chat Application is a practical demonstration of how two or more programs on a network can exchange data in real time using standard networking protocols.

This project was undertaken as part of the Computer Networks course to understand and implement the core concepts of socket programming, the Client-Server model, TCP/IP protocol, and multi-threaded programming. The Cisco Packet Tracer simulation is included to show the underlying network infrastructure on which such an application would run.

2.1 Objectives

The main objectives of this project are as follows:

- To implement a working chat application using Python 3 socket programming
- To demonstrate the Client-Server architecture through a practical program
- To handle multiple simultaneous clients using the threading module
- To simulate the network topology in Cisco Packet Tracer 8.x
- To verify network connectivity using ping tests and PDU packet simulation
- To understand the OSI model layers as they apply to this application

2.2 Scope

The project covers the design, coding, testing, and simulation of a basic multi-user chat system. It runs on a local machine using the loopback address (127.0.0.1) for testing purposes, but can easily be extended to work across a real local area network by changing the IP address to the server machine's actual IP.

Made with GAMMA

Background Theory

3.1 Socket Programming

A socket is a software abstraction that provides an endpoint for communication between two processes over a network. The socket API was originally developed for Unix and has since become the standard interface for network programming across all major operating systems. In Python, the socket module wraps the underlying OS socket calls and makes them accessible through a simple object-oriented interface.

A socket is identified by two properties: an IP address, which identifies the machine, and a port number, which identifies the specific process or application on that machine. Our server uses port 12345 to listen for incoming connections.

3.2 TCP Protocol

TCP stands for Transmission Control Protocol. It is a connection-oriented protocol that operates at Layer 4 (Transport Layer) of the OSI model. TCP provides reliable, ordered, and error-checked delivery of data between applications. It achieves this through the following mechanisms:

- Three-way handshake — SYN, SYN-ACK, ACK — to establish a connection
- Sequence numbers to ensure data arrives in the correct order
- Acknowledgement (ACK) messages to confirm delivery
- Retransmission of lost packets

For a chat application, TCP is the correct choice because every message must arrive completely and in order. An alternative protocol, UDP (User Datagram Protocol), is faster but does not guarantee delivery or ordering, making it unsuitable for chat.

3.3 Client-Server Architecture

In the Client-Server model, one process (the server) waits for connection requests from other processes (clients). The server is always running and ready to accept new connections, while clients initiate the communication. In our chat application, the server acts as a central hub: every message sent by a client is received by the server and forwarded to all other connected clients. This is known as broadcasting.

3.4 Multi-Threading

A thread is the smallest unit of a process that can be scheduled and executed independently. Python's threading module allows a program to run multiple threads concurrently. In our server, each time a new client connects, a dedicated thread is created for that client using threading.Thread(). This allows the server to handle many clients at the same time without blocking — when Thread 1 is waiting for Alice to send a message, Thread 2 can still handle Bob's messages independently.

3.5 OSI Model and This Project

Layer	Name	Python / Code	Cisco PT
7	Application	Chat logic — send/receive messages	—
6	Presentation	encode('utf-8') / decode('utf-8')	—
5	Session	socket.connect() / accept()	—
4	Transport	socket.SOCK_STREAM (TCP), port 12345	TCP segment in PDU
3	Network	IP address 127.0.0.1	IP packet in PDU
2	Data Link	Handled by OS	Ethernet frame via Switch0
1	Physical	Network card / loopback	Copper cable in PT

Made with GAMMA

System Design

4.1 Architecture Overview

The system consists of two programs — server.py and client.py. The server runs continuously and manages all client connections. Each client connects to the server, provides a nickname, and can then send and receive messages. The server maintains two lists: one for client socket objects and one for their nicknames. When a message arrives from any client, the server calls its broadcast() function which sends the message to every client in the list.

4.2 Network Topology (Cisco Packet Tracer)

In the Cisco Packet Tracer simulation, the following devices represent the physical network on which the chat application would run:

Device	IP Address	Subnet Mask	Role
PC-Server	192.168.1.1	255.255.255.0	Runs server.py, port 12345
PC-Alice	192.168.1.2	255.255.255.0	Client 1 — runs client.py
PC-Bob	192.168.1.3	255.255.255.0	Client 2 — runs client.py
PC-Charlie	192.168.1.4	255.255.255.0	Client 3 — runs client.py
Switch0 (2960)	N/A	N/A	Connects all PCs — Layer 2 device

All PCs are connected to Switch0 using Copper Straight-Through cables. PC-Alice connects on Fa0/1, PC-Bob on Fa0/2, PC-Charlie on Fa0/3 and PC-Server on Fa0/4.

4.3 Message Flow

The sequence below describes how a message sent by Alice reaches Bob:

1. Alice types a message in her terminal and presses Enter
2. The send() function reads the input, prefixes her nickname, and calls .encode('utf-8') to convert the string to bytes
3. socket.send() transmits the bytes to the server over the established TCP connection
4. The server's handle_client() thread for Alice calls socket.recv(1024) which returns the incoming bytes
5. The server calls broadcast(), which loops through the clients[] list and calls socket.send() on each socket
6. Bob's receive() thread calls socket.recv(1024), decodes the bytes using .decode('utf-8'), and prints the message

Made with GAMMA

Source Code

5.1 server.py — Chat Server

server.py

```python
import socket
import threading

HOST = '127.0.0.1' # IP address of this machine
PORT = 12345 # Port number the server listens on

clients = [] # stores connected client sockets
nicknames = [] # stores nicknames of connected clients

# broadcast() sends a message to every connected client
def broadcast(message):
 for client in clients:
 try:
 client.send(message)
 except:
 # if send fails the client has disconnected
 remove_client(client)

# remove_client() cleans up when a client disconnects
def remove_client(client):
 if client in clients:
 index = clients.index(client)
 name = nicknames[index]
 clients.remove(client)
 nicknames.remove(name)
 client.close()
 broadcast(f'{name} has left the chat.'.encode('utf-8'))
 print(f'[INFO] {name} disconnected.')

# handle_client() runs in its own thread for each client
def handle_client(client):
 while True:
 try:
 message = client.recv(1024)
 broadcast(message)
 except:
 remove_client(client)
 break

# start_server() creates the socket and accepts connections
def start_server():
 server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
 server.bind((HOST, PORT))
 server.listen()
 print(f'Server running on {HOST}:{PORT}')
 print('Waiting for clients to connect...')

 while True:
 client, address = server.accept()
 print(f'Connection from {address}')

 # ask the client for their nickname first
 client.send('NICK'.encode('utf-8'))
 nickname = client.recv(1024).decode('utf-8')
 nicknames.append(nickname)
 clients.append(client)
 print(f'Nickname: {nickname}')

 broadcast(f'{nickname} joined the chat!'.encode('utf-8'))
 client.send('Connected to the server!'.encode('utf-8'))

 # create a dedicated thread for this client
 thread = threading.Thread(target=handle_client, args=(client,))
 thread.daemon = True
 thread.start()

start_server()
```

Made with GAMMA

Client Code

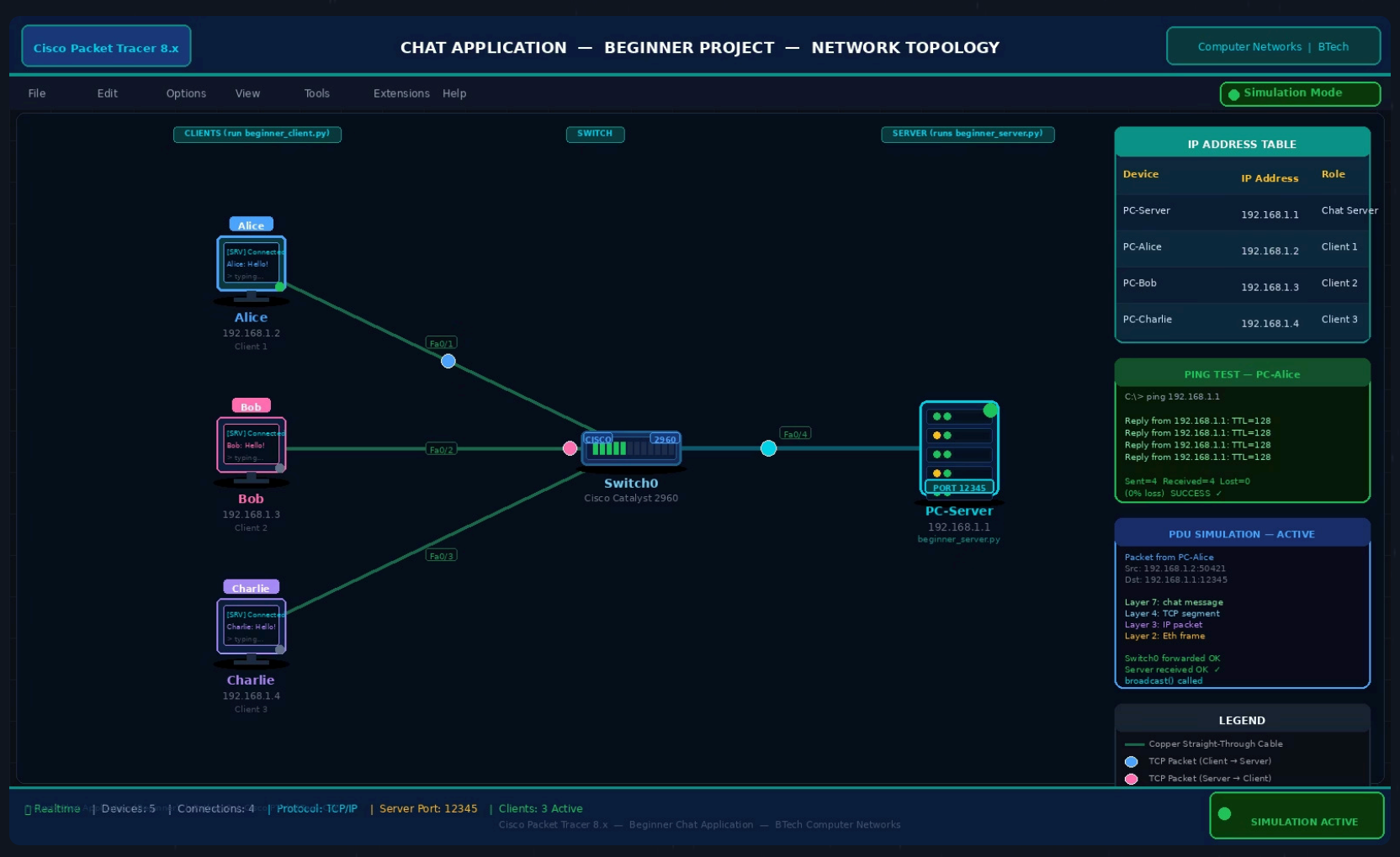
5.2 client.py — Chat Client

client.py

```
import socket
import threading

HOST = '127.0.0.1' # must match the server
PORT = 12345 # must match the server

nickname = input('Enter your nickname: ')

# create socket and connect to server
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

# receive() runs in a background thread
# it listens for incoming messages from the server
def receive():
 while True:
 try:
 message = client.recv(1024).decode('utf-8')
 if message == 'NICK':
 # server is asking for our nickname
 client.send(nickname.encode('utf-8'))
 else:
 print(message)
 except:
 print('Disconnected from server.')
 client.close()
 break

# send() runs on the main thread
# it reads keyboard input and sends it to the server
def send():
 while True:
 try:
 message = f'{nickname}: {input("")}'
 client.send(message.encode('utf-8'))
 except:
 client.close()
 break

# start both threads at the same time
receive_thread = threading.Thread(target=receive)
receive_thread.daemon = True
receive_thread.start()

send_thread = threading.Thread(target=send)
send_thread.daemon = True
send_thread.start()
```

Made with GAMMA

Cisco Packet Tracer Setup



6.1 Steps to Build the Topology

The following steps were carried out to create and test the network simulation in Cisco Packet Tracer 8.x:

Step 1 — Add Devices

Four PC end devices and one Cisco Catalyst 2960 Switch were dragged from the device panel onto the workspace. The PCs were renamed PC-Server, PC-Alice, PC-Bob and PC-Charlie.

Step 2 — Connect Devices

Copper Straight-Through cables were used to connect each PC to the switch. PC-Alice was connected to Fa0/1, PC-Bob to Fa0/2, PC-Charlie to Fa0/3 and PC-Server to Fa0/4. After a few seconds, the link indicators turned green confirming the connections were active.

Step 3 — Assign IP Addresses

Each PC was configured with a static IP address through Desktop → IP Configuration. The subnet mask 255.255.255.0 was used for all devices, placing them on the same network (192.168.1.0/24).

Step 4 — Connectivity Test (Ping)

To verify that the network was correctly configured, a ping test was run from PC-Alice to PC-Server using the Command Prompt in the Desktop tab. The result showed four successful replies with 0% packet loss, confirming that Alice could reach the server across the switch.

```
PC-Alice → Desktop → Command Prompt

C:\> ping 192.168.1.1

Pinging 192.168.1.1 with 32 bytes of data:
Reply from 192.168.1.1: bytes=32 time<1ms TTL=128
Reply from 192.168.1.1: bytes=32 time<1ms TTL=128
Reply from 192.168.1.1: bytes=32 time<1ms TTL=128
Reply from 192.168.1.1: bytes=32 time<1ms TTL=128

Ping statistics for 192.168.1.1:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss)
```

Step 5 — PDU Simulation

The simulation was switched to Simulation Mode using the clock icon. A Simple PDU was added from PC-Alice (source) to PC-Server (destination). The Play button was used to animate the packet travelling from Alice through Switch0 to the Server. Clicking on the packet envelope displayed the full PDU detail window showing all OSI layers including the TCP segment at Layer 4 with destination port 12345, and the IP packet at Layer 3 with destination address 192.168.1.1.

6.2 Relationship Between Cisco PT and Python Code

Python Code	Cisco PT Element	What it means
HOST = '127.0.0.1'	PC-Server IP: 192.168.1.1	Server fixed address
PORT = 12345	PDU destination port: 12345	Same port number
server.bind((HOST,PORT))	IP Configuration on PC-Server	Server assigned an address
client.connect((HOST,PORT))	Cable connecting PC-Alice to Switch	Creating a connection
socket.SOCK_STREAM	TCP segment at Layer 4 in PDU	TCP protocol in use
client.send(message)	Packet travelling in simulation	Data sent over network
broadcast() to all clients	PDU received by all PCs	Message reaches everyone
Thread per client	Multiple PDUs from different PCs	Multiple clients at once

Made with GAMMA

Results and Output

7.1 Python Program Output

The following output was observed when the server and two clients were run simultaneously on the same machine:

Terminal 1 — python server.py

```
--- Terminal 1: server.py ---
Server running on 127.0.0.1:12345
Waiting for clients to connect...
Connection from ('127.0.0.1', 50421)
Nickname: Alice
Connection from ('127.0.0.1', 50422)
Nickname: Bob
Alice: Hello Bob, are you there?
Bob: Yes Alice, I can see your message!
Alice: Great, the project is working.
```

Terminal 2 — python client.py

```
--- Terminal 2: client.py (Alice) ---
Enter your nickname: Alice
Connected to the server!
Alice joined the chat!
Bob joined the chat!
Alice: Hello Bob, are you there?
Bob: Yes Alice, I can see your message!
Alice: Great, the project is working.
```

Terminal 3 — python client.py

```
--- Terminal 3: client.py (Bob) ---
Enter your nickname: Bob
Connected to the server!
Bob joined the chat!
Alice: Hello Bob, are you there?
Bob: Yes Alice, I can see your message!
Alice: Great, the project is working.
```

7.2 Cisco PT Results

- All four PCs successfully pinged each other with 0% packet loss
- PDU simulation showed the packet travelling from PC-Alice through Switch0 to PC-Server
- The PDU details window confirmed TCP protocol at Layer 4 with destination port 12345
- Multiple PDUs from different client PCs to the server demonstrated the multi-client scenario

Discussion

The project successfully demonstrates the practical implementation of a Client-Server chat system using Python socket programming. A few observations worth noting from the testing are described below.

The use of Python's threading module was essential for making the server handle multiple clients without blocking. During testing, it was observed that when a third client (Charlie) was connected while Alice and Bob were already chatting, all three users could exchange messages without any delay or interference. This confirms that each thread operates independently.

One limitation noticed during testing is that if the server is stopped while clients are still connected, the clients receive a connection error on their next recv() call. The current implementation handles this gracefully by catching the exception and printing a disconnection message, but in a production-level application it would be better to notify the user more clearly and offer a reconnect option.

The Cisco Packet Tracer simulation, while limited to a single PDU simulation rather than real TCP data, effectively visualises the layer-by-layer encapsulation of data as it travels from a client PC to the server PC through the switch. The PDU detail panel is particularly useful for demonstrating how the application-level data (our chat message) gets wrapped in TCP and IP headers as it travels down the OSI stack.

Made with GAMMA

Conclusion

This project has successfully implemented a Chat Application that demonstrates several important Computer Networks concepts in a practical, working system. The Python implementation covers socket programming, the Client-Server model, TCP/IP communication, and multi-threaded programming — all using only the standard library with no external dependencies.

The Cisco Packet Tracer simulation provides a complementary visual representation of the network infrastructure, showing how IP addressing, switching, and packet transmission work at the hardware and protocol level. Together, the two components cover the complete picture from the physical cable (Layer 1) through to the running application (Layer 7).

The project can be further extended by adding features such as a graphical user interface using tkinter, user authentication with a username and password, message encryption using AES or RSA, support for file transfer between users, and the ability to create private chat rooms within the same server.

References

1. Forouzan, B.A. (2012). *Data Communications and Networking, 5th Edition*. New York: McGraw-Hill Education.
2. Tanenbaum, A.S. and Wetherall, D.J. (2011). *Computer Networks, 5th Edition*. New Jersey: Pearson Education.
3. Python Software Foundation. (2024). *socket — Low-level networking interface*. Python 3 Documentation. Retrieved from **https://docs.python.org/3/library/socket.html**
4. Python Software Foundation. (2024). *threading — Thread-based parallelism*. Python 3 Documentation. Retrieved from **https://docs.python.org/3/library/threading.html**
5. Cisco Systems. (2023). *Cisco Packet Tracer 8.x User Guide*. Cisco Networking Academy.
6. Stevens, W.R., Fenner, B. and Rudoff, A.M. (2003). *Unix Network Programming, Volume 1: The Sockets Networking API, 3rd Edition*. Boston: Addison-Wesley.

Made with GAMMA